Sicherheitsanalyse: Bauloo Styles

RAT- und Malware-Prüfung der Minecraft-Fabric-Mod (Version 1.6.0, MC 1.21.11) sowie Abgleich der offiziellen JAR-Dateien mit dem veröffentlichten Quellcode

Fertiggestellt	20.10.2026, 00:53 Uhr MESZ (UTC+2)
Art der Prüfung	**Unabhängige Prüfung aus eigenem Interesse.** Keine Verbindung zum Ersteller der Mod (siehe Abschnitt 10)
Geprüft	Repository **bauloo-styles-main** (Quellcode), Repository **bauloo-styles-cloud-main** (Releases, Packs, Tools), zwei JAR-Dateien von der offiziellen Website
Methode	Statische Code-Analyse, Bytecode-Analyse, Dekompilierung (Vineflower 1.11.1), reproduzierbarer Build aus dem Quellcode und Byte-für-Byte-Vergleich
Erstellt von	Riyu(FPS) in Kooperation mit Claude Code (Opus 5.5)
Kontakt	Discord @RiyuFPS · GitHub github.com/ryufps · E-Mail JulianMink.2007@gmail.com

Ergebnis: Kein RAT, kein Stealer, keine Malware gefunden.

Die offizielle JAR-Datei **bauloo-styles-1.6.0-mc1.21.11-auto-update.jar** ist im Programmcode **bytegleich** mit dem, was aus dem veröffentlichten Quellcode gebaut wird: alle 164 Klassendateien haben identische SHA-256-Prüfsummen. Der Quellcode, den der Autor als Nachweis veröffentlicht hat, ist also genau der Code, der auf deinem PC läuft.

Die zweite Datei **bauloo-styles-1.6.0-mc1.21.11.jar** unterscheidet sich nur in 4 Klassen: dort ist das Spotify-Feature entfernt. Sie enthält keinen zusätzlichen Code.

Gefunden wurden aber einige **Datenschutz-Punkte** (Telemetrie mit Minecraft-Namen, Fernsperre, Präsenz-Abfrage). Sie sind in Abschnitt 7 beschrieben, sind aber keine Schadfunktionen.

Unabhängige Prüfung

Riyu(FPS) hat die Mod gemeinsam mit Claude Code **aus eigenem Interesse und unabhängig vom Ersteller** untersucht. Es besteht keine Zusammenarbeit, kein Auftrag und keine sonstige Verbindung mit dem Ersteller der Mod oder seinen Projekten. Das Ergebnis gilt nur für die geprüften Dateien zum Prüfzeitpunkt; für künftige Versionen gibt es keine Gewähr (Disclaimer in Abschnitt 10).

Inhalt

- 1. Untersuchte Dateien
- 2. Vorgehen
- 3. Abgleich offizielle JARs und Repository (Kernergebnis)
- 4. Unterschied der beiden Website-Varianten
- 5. Code-Analyse: Prüfung auf RAT- und Stealer-Merkmale
- 6. Cloud-Repository, Packs und Build-Umgebung
- 7. Datenschutz-Hinweise (keine Malware)
- 8. Grenzen der Analyse
- 9. Empfehlungen
- 10. Unabhängigkeit und Haftungsausschluss (Disclaimer)
- 11. Kontakt
- Anhang A: Prüfsummen
- Anhang B: Werkzeuge und Nachvollziehbarkeit

1. Untersuchte Dateien

Datei / Ordner	Inhalt	Umfang
bauloo-styles-1.6.0-mc1.21.11-auto-update.jar	Offizielle Mod-Datei von der Website (Variante mit Spotify)	10.123.495 Bytes
bauloo-styles-1.6.0-mc1.21.11.jar	Offizielle Mod-Datei von der Website (Variante ohne Spotify)	10.114.133 Bytes
bauloo-styles-main/	Quellcode der Mod: Java, Ressourcen, Gradle-Build	111 Java-Dateien, 21.038 Zeilen; 571 Bilder
bauloo-styles-cloud-main/release/	Fertige JARs für 12 Minecraft-Versionen, je 2 Varianten, plus version.json	24 JARs
bauloo-styles-cloud-main/packs/	Texture-Packs, die die Mod bei Bedarf herunterlädt	301 ZIPs
bauloo-styles-cloud-main/previews/	Vorschaubilder der Packs	301 PNGs
bauloo-styles-cloud-main/tools/	Veröffentlichungs-Skript des Autors (stylescloud.py) und kompilierte .pyc	2 Dateien

Die beiden Website-JARs sind bytegleich mit den Dateien im Cloud-Repository:

Website-Datei	identisch mit	SHA-256
…-auto-update.jar	release/bauloo-styles.jar	`20684bceb237774ae1bc9b208216b057d3f93fbebdf9b4cfa72cdb5856d6c19f`
…-mc1.21.11.jar	release/bauloo-styles-no-auto-update.jar	`3056d0be0cb5e4daa3d3b7830be551036a0e4c3b4976e922fe1b531cafa23b7a`

2. Vorgehen

Die Prüfung lief in mehreren unabhängigen Schritten. Keine der untersuchten Dateien wurde ausgeführt, mit Ausnahme des Gradle-Builds aus dem Quellcode. Dessen Build-Skripte wurden vorher gelesen; der Build lief in einer isolierten Kopie mit eigenem Gradle-Cache.

- **Quellcode-Review:** Suche nach typischen Schadcode-Mustern (Prozessstart, Nachladen von Code, Zugriff auf Browser-, Discord- oder Launcher-Daten, Session-Token, Tastatur/Bildschirm, Verschlüsselung/Verschleierung) und manuelles Lesen aller Klassen mit Netzwerk- oder Dateizugriff.
- **Bytecode-Analyse:** Eigenes Skript liest die Konstanten-Pools aller Klassen in allen 26 JARs und listet jede aufgerufene Methode und jeden String. Dabei wird kein Code geladen oder ausgeführt.
- **Strukturprüfung der JARs:** Fremde Klassen, eingebettete JARs, DLL/EXE-Dateien, Manifest und Fabric-Einstiegspunkte.
- **Dekompilierung:** Beide Website-JARs mit Vineflower 1.11.1 (Prüfsumme gegen Maven Central verifiziert).
- **Reproduzierbarer Build:** Der Quellcode wurde mit dem mitgelieferten Gradle-Wrapper gebaut (Gradle 9.6.1, Fabric Loom 1.17.16, JDK 21.0.12). Die selbst gebaute JAR wurde auf Ebene einzelner Dateien (SHA-256) und auf Ebene des dekompilierten Codes mit den offiziellen JARs verglichen.
- **Inhalte:** Alle 301 Packs gegen die Prüfsummen in index.json, Dateitypen, Pfad-Tricks (Zip-Slip), Magic Bytes und angehängte Daten in Bildern; .pyc gegen .py; Gradle-Wrapper gegen die offiziellen Prüfsummen von gradle.org.

3. Abgleich offizielle JARs und Repository (Kernergebnis)

Der direkte Vergleich von dekompiliertem Code mit dem Original-Quellcode ist nur grob möglich: Kommentare fehlen, Konstanten setzt der Compiler direkt ein (z. B. wird `MOD_ID` zu `"bauloo_styles"`), und Minecraft-Methoden tragen in der JAR Zwischennamen wie `method_25394` statt `render`. Ein solcher grober Vergleich zeigte nur dieses

erwartbare Rauschen.

Deshalb wurde der Quellcode zusätzlich selbst gebaut. Stimmt der gebaute Bytecode mit der offiziellen JAR überein, ist bewiesen, dass die JAR aus genau diesem Quellcode stammt. Ergebnis:

Vergleich	Ergebnis
Klassendateien (.class), offizielle auto-update-JAR vs. eigener Build	**164 von 164 bytegleich** (identische SHA-256)
Dekompilierter Code, offizielle auto-update-JAR vs. eigener Build	**keine Unterschiede** in 111 Java-Dateien (21.892 Zeilen)
Ressourcen außerhalb von resourcepacks/	772 von 779 bytegleich; 7 abweichend, siehe unten
Dateien nur in der offiziellen JAR	1.170 Dateien in resourcepacks/ (97 eingebaute Texture-Packs)
Dateien nur im eigenen Build	keine

Erklärung der Abweichungen:

- `fabric.mod.json`: Nur die Versionsangabe unterscheidet sich (das offizielle Build-Skript hängt die MC-Version an). Einstiegspunkte, Mixins und Abhängigkeiten sind gleich.
- 5× `respawn_anchor_N.json` und `tornado.png.mcmeta`: inhaltlich identisch (als JSON verglichen). Die Bytes unterscheiden sich nur durch Zeilenenden (Windows-CRLF gegenüber Unix-LF).
- `resourcepacks/`: Die 97 eingebauten Packs stehen laut README bewusst nicht im Repository (Rechte der Pack-Ersteller). Sie wurden separat geprüft: nur Bilder, JSON und .mcmeta, dazu drei harmlose Windows-Systemdateien (desktop.ini, 2× thumbs.db). Keine Klassen, keine ausführbaren Dateien.

```
diff official-auto-update/fabric.mod.json  repo-build/fabric.mod.json
<   "version": "1.6.0+mc1.21.11",
>   "version": "1.6.0",
<   "minecraft": "1.21.11",
>   "minecraft": ">=1.21.11",
```

4. Unterschied der beiden Website-Varianten

Die Dateinamen deuten auf einen Unterschied beim automatischen Update hin. Das stimmt nicht: Beide Varianten enthalten exakt denselben Update-Code, und der prüft seit Version 1.6.0 nur, ob eine neue Version existiert. Er lädt nichts herunter und ersetzt keine Dateien.

Der tatsächliche Unterschied betrifft nur 4 Klassen, alle zum Spotify-Feature:

Klasse	auto-update (= Repository)	ohne Zusatz
SpotifySmtc	Liest über JNA und die Windows-Mediensteuerung (combase/WinRT) Titel, Interpret, Position und Cover der Spotify-Wiedergabe	Leerer Platzhalter, gibt immer null zurück
SpotifyNowPlaying	Hintergrund-Thread; Fallback liest Fenstertitel (User32.EnumWindows) und filtert auf Prozesse namens spotify.*	AVAILABLE = false, start() tut nichts
SpotifySmtc$Info	Datenklasse	inhaltlich gleich (nur Compiler-Details abweichend)
BaulooStylesConfigScreen	Menü enthält Spotify-Kachel und -Schalter	Spotify-Kachel und -Schalter entfernt

Vermutlich gibt es die Variante ohne Spotify, weil Virenscanner Fenster-Enumeration und native Windows-Aufrufe gern als verdächtig einstufen. Die Spotify-Funktion ist standardmäßig **aus** (`spotifyHud = false`). Solange sie aus ist, werden weder Fenster gelesen noch Anfragen an Deezer/iTunes geschickt.

5. Code-Analyse: Prüfung auf RAT- und Stealer-Merkmale

Typische Merkmale von Minecraft-Mod-Malware (Session-Stealer, Discord-Token-Grabber, RATs) und der Befund im Code. Die Prüfung erfolgte im Quellcode und zusätzlich im Bytecode aller 26 JARs.

Merkmal	Befund
Programme starten (Runtime.exec, ProcessBuilder)	Nicht vorhanden
Code nachladen (ClassLoader, defineClass, URLClassLoader, Script-Engines)	Nicht vorhanden
Reflection / Class.forName	Nur für optionale Kompatibilität mit den Mods *Blur* (BlurConfig) und *Sodium Extra*
Minecraft-Session-Token (getAccessToken / getSessionId)	Nicht vorhanden. Gelesen werden nur **Name** (method_1676) und **UUID** (method_44717)
Browser-Daten, Cookies, Passwörter (Login Data, Local State)	Nicht vorhanden
Discord-Token (leveldb), Launcher-Konten (launcher_accounts.json)	Nicht vorhanden
Krypto-Wallets	Nicht vorhanden
Keylogger, Bildschirmfotos (Robot, GlobalScreen)	Nicht vorhanden
Zwischenablage	Nur im Menü: Kopieren/Einfügen von Preset-Codes durch den Nutzer
Verschleierung, Base64, Verschlüsselung	Nicht vorhanden; nichts ist obfuskiert
Native Bibliotheken / DLLs in der JAR	Keine; JNA wird von Minecraft mitgeliefert und nur für Spotify genutzt
Eingebettete JARs (Jar-in-Jar)	Keine
Eingehende Verbindungen / Fernsteuerung	Kein Fernzugriff; lokaler Server nur auf 127.0.0.1 (siehe 5.3)
Selbst-Update / Nachladen von Programmcode	Nein, nur Versionsprüfung mit Hinweis im Menü

5.1 Netzwerkverbindungen

Vollständige Liste aller im Bytecode vorhandenen Ziele:

Ziel	Zweck	Gesendete Daten
raw.githubusercontent.com, api.github.com, cdn.jsdelivr.net (Repo Bauloo0/bauloo-styles-cloud)	Style-Liste, Packs, Vorschaubilder, Versionsprüfung, Sperrliste	Keine persönlichen Daten
bauloo-styles-cloud.bauloo-styles-cloud-worker.workers.dev (Cloudflare-Worker des Autors)	Spiegel für Downloads, Statistik, Tab-Icons, Presets, Feedback, VIP-Funktionen	Minecraft-Name, Hash der Geräte-ID, UUIDs (Details in Abschnitt 7)
textures.minecraft.net	Skin-/Cape-Texturen	Textur-Hash
api.deezer.com, itunes.apple.com	Album-Cover für die Spotify-Anzeige (nur wenn aktiviert)	Interpret und Songtitel
baulooprac.xyz	Feedback-Seite, wird im Browser geöffnet	Einmal-Token in der URL
127.0.0.1 (lokal)	Upload-Seite für eigene Packs	Bleibt auf dem PC

5.2 Dateizugriffe

Die Mod liest und schreibt nur in diesen Ordnern:

- `.minecraft/config/bauloo-styles/`: Cache, Einstellungen, eigene Uploads, Capes

- `.minecraft/resourcepacks/`: heruntergeladene und eigene Style-Packs (Präfix bauloo_)
- `%USERPROFILE%/.bauloo-styles/`: zufällige Geräte-ID (device-id.txt), optional ein Publish-Schlüssel des Nutzers
- Temporäre Dateien im System-Temp-Ordner beim Download

Verzeichnisse werden nur innerhalb dieser Ordner durchsucht (Files.walk / Files.list). Andere Bereiche der Festplatte werden nicht gelesen.

5.3 Lokaler Webserver (Upload-Seite)

Ein kleiner HTTP-Server in `StylesUploadServer` startet nur, wenn der Nutzer die Upload-Seite öffnet. Er ist korrekt abgesichert:

- Bindet nur an die Loopback-Adresse (`InetAddress.getLoopbackAddress()`); aus dem Netzwerk nicht erreichbar.
- Zufälliger Port und zufälliges 128-Bit-Token (SecureRandom), Vergleich in konstanter Zeit.
- Prüft Host- und Origin-Header; schützt gegen DNS-Rebinding und fremde Webseiten.
- Beendet sich nach 30 Minuten ohne Nutzung.

5.4 Downloads und Verarbeitung von Packs

- Jedes Pack wird gegen Größe und SHA-256 aus index.json geprüft; ein Spiegelserver kann deshalb keine anderen Dateien unterschieben.
- Aus dem ZIP werden nur Einträge unter `assets/` (bzw. `assets/minecraft/optifine/sky/` für Himmel) übernommen; Pfade mit `..` oder führendem `/` werden verworfen (Schutz vor Zip-Slip).
- Größenlimits: 95 MB pro Pack, 400 MB entpackt, 2 MB pro Vorschaubild.
- Die Remote-Konfiguration in index.json ist eingeschränkt: Die Server-Adresse muss dem Muster `https://*.*.workers.dev` entsprechen, Kategorie-Regeln dürfen nur Pfade unter `assets/` erzeugen.

5.5 Start vor dem Spiel (preLaunch)

Die Mod registriert einen `preLaunch`-Einstiegspunkt. So etwas nutzt Malware gern, um vor Minecraft aktiv zu werden. Hier startet er nur einen Hintergrund-Thread, der `release/version.json` von GitHub liest und die Versionsnummer vergleicht. Es wird nichts heruntergeladen, geschrieben oder ausgeführt.

6. Cloud-Repository, Packs und Build-Umgebung

Bestandteil	Befund
24 Release-JARs (12 MC-Versionen × 2 Varianten)	Keine fremden Klassen, keine DLL/EXE, keine eingebetteten JARs. Die Versionen für andere MC-Versionen haben kleine Kompatibilitätsklassen (z. B. WorldHook, compat/Click, ShieldContext). Der Bytecode-Scan zeigt dort keine zusätzlichen sensiblen Aufrufe und keine zusätzlichen URLs außer der jeweiligen version.json.
301 Texture-Packs	Alle passen exakt zu Größe und SHA-256 in index.json. Inhalt: PNG, JSON, .mcmeta, .properties. Dazu Windows-Reste (desktop.ini, Thumbs.db) und zwei deaktivierte .properties-Dateien. Keine ausführbaren Dateien, keine Pfad-Tricks.
301 Vorschaubilder	Gültige PNG-Dateien ohne angehängte Daten nach dem Bildende.
tools/stylescloud.py	Veröffentlichungs-Skript des Autors. Ruft nur git auf (pull/add/commit/push) im eigenen Repository. Wird von der Mod nicht verwendet.
tools/__pycache__/...pyc	Kompilierte ältere Fassung desselben Skripts (Bytecode verglichen, ohne Ausführung): keine Funktionen oder Namen, die nicht auch im .py stehen.
gradle-wrapper.jar	Offizieller Gradle-Wrapper; SHA-256 stimmt mit den von gradle.org veröffentlichten Prüfsummen für Gradle 8.7/8.8 überein.
build.gradle / settings.gradle	Standard-Fabric-Build ohne eigene Tasks; Abhängigkeiten nur von Fabric, Mojang und TerraformersMC (ModMenu).

7. Datenschutz-Hinweise (keine Malware)

Die folgenden Punkte sind keine Schadfunktionen, du solltest sie aber kennen. Sie gelten für beide Website-Varianten gleichermaßen.

Telemetrie mit Minecraft-Namen, nicht abschaltbar

Alle 10 Minuten sendet `StylesStats` den **Minecraft-Namen** und einen SHA-256-Hash einer zufällig erzeugten Geräte-ID an den Worker des Autors (`/ping`). Das README beschreibt das als „anonymous usage stats“. Durch den Namen ist das nicht anonym. Einen Schalter dafür gibt es im Code nicht. Die Geräte-ID ist zufällig und enthält keine Hardware-Merkmale.

Präsenz im Tab und Abfrage anderer Spieler

Auf einem Server sendet die Mod regelmäßig die eigene Spieler-UUID an `/presence`. Außerdem schickt sie die UUIDs der Spieler in der Tab-Liste (bis zu 100) an `/presence/lookup`, um deren Icons anzuzeigen. Der Server des Autors erfährt dadurch, wer mit wem auf einem Server ist. Mit „Show in tab“ aus (Standard: an) wird nur das eigene Icon versteckt; Ping und Abfrage laufen weiter.

Fernsperre

Über `blocked.json` im Cloud-Repository kann der Autor einzelne Accounts sperren. Dann zeigt die Mod kein Menü mehr und deaktiviert ihre eigenen Resource-Packs. Es wird nichts gelöscht und nichts anderes auf dem PC verändert.

Fernkonfiguration

Über `index.json` kann der Autor die Server-Adresse und neue Style-Kategorien ändern, ohne neue Mod-Version. Beides ist im Code eng eingegrenzt (siehe 5.4); Programmcode lässt sich darüber nicht einschleusen.

Spotify → Deezer/iTunes (nur Variante mit Spotify, standardmäßig aus)

Ist die Spotify-Anzeige eingeschaltet, werden Interpret und Titel des laufenden Songs an Deezer bzw. Apple geschickt, um ein Cover zu finden, falls Windows keins liefert.

Veralteter Beschreibungstext

In `fabric.mod.json` steht noch „Keeps itself up to date“. Der Code aktualisiert sich seit 1.6.0 aber nicht mehr selbst. Auch der Dateiname „auto-update“ ist irreführend (siehe Abschnitt 4).

> **Was der Code nicht klären kann**
>
> Was der Worker des Autors mit Namen, UUIDs und Geräte-Hashes macht, ist von außen nicht prüfbar. Der Code-Kommentar sagt, nur der Autor sehe die Namen und es werde keine IP gespeichert; das lässt sich nicht verifizieren.

8. Grenzen der Analyse

- **Statische Analyse:** Die Mod wurde nicht in Minecraft gestartet und ihr Netzwerkverkehr wurde nicht mitgeschnitten. Durch den bytegleichen Build ist aber gesichert, dass genau der gelesene Quellcode läuft.
- **Server-Seite:** Der Cloudflare-Worker und die Website baulooprac.xyz sind nicht Teil der Repositories und wurden nicht untersucht.
- **Zukünftige Versionen:** Die Analyse gilt nur für die Dateien mit den Prüfsummen in Anhang A. Neue Versionen müssen erneut geprüft werden. Weil die Mod sich nicht selbst aktualisiert, ändert sich der Code

nur, wenn du selbst eine neue Datei installierst.

- **JARs für andere MC-Versionen** (release/mc/…) wurden per Bytecode-Scan geprüft, aber nicht nachgebaut und nicht Byte für Byte verglichen; sie sind für MC 1.21.11 nicht relevant.
- **Bildinhalte** der Packs wurden auf Dateityp und angehängte Daten geprüft. Minecraft verarbeitet sie nur als Textur; ausführbar sind sie nicht.

9. Empfehlungen

- Nutze nur Dateien, deren SHA-256 mit Anhang A übereinstimmt. Unter Windows: `Get-FileHash <Datei>` in PowerShell.
- Brauchst du die Spotify-Anzeige nicht, nimm die Variante **ohne** „auto-update“ im Namen. Sie enthält weniger Code mit Systemzugriff. Die Telemetrie ist in beiden Varianten gleich.
- Willst du nicht im Tab angezeigt werden, schalte „Show in tab“ in den Extras aus. Beachte, dass Ping und Abfrage weiterlaufen.
- Wer die Telemetrie ganz vermeiden will, kann die Domain `bauloo-styles-cloud-worker.workers.dev` in der Firewall oder hosts-Datei sperren. Cloud-Styles funktionieren dann weiter über GitHub; Statistik, Tab-Icons, Presets und VIP-Funktionen nicht.
- Bei künftigen Updates: Quellcode und JAR erneut vergleichen, am besten wieder über einen eigenen Build.

10. Unabhängigkeit und Haftungsausschluss (Disclaimer)

Unabhängigkeit

Diese Prüfung ist eine **unabhängige Untersuchung aus eigenem Interesse**. Riyu(FPS) hat die Mod gemeinsam mit Claude Code (Opus 5.5), einem KI-Assistenten von Anthropic, unabhängig vom Ersteller untersucht. Grundlage waren ausschließlich öffentlich verfügbare Dateien: die beiden GitHub-Repositories und die Downloads von der offiziellen Website.

- Riyu(FPS) ist **nicht mit dem Ersteller der Mod verbunden** (not affiliated), weder mit dem Ersteller selbst noch mit seinem Discord-Server, seiner Website oder seinen sonstigen Projekten.
- Es besteht **keine Zusammenarbeit, kein Auftrag und keine sonstige Verbindung** mit dem Ersteller.
- Der Ersteller war an der Prüfung nicht beteiligt und hatte keinen Einfluss auf Inhalt oder Ergebnis dieses Berichts.
- Dieser Bericht ist keine Werbung für die Mod und keine Empfehlung, sie zu nutzen.

Haftungsausschluss

- **Momentaufnahme:** Das Ergebnis gilt ausschließlich für die in Anhang A per SHA-256 benannten Dateien, Stand der Prüfung: 20.10.2026, 00:53 Uhr MESZ. Andere Versionen, andere Download-Quellen oder veränderte Dateien sind davon nicht abgedeckt.
- **Keine Gewähr für die Zukunft:** Es wird **keine Gewährleistung** dafür übernommen, dass in künftigen Versionen oder Updates der Mod kein Schadcode eingebaut wird. Der Ersteller kann den Code jederzeit ändern. Das gilt auch für seine Server (Cloudflare-Worker, GitHub-Repository, Website), deren Verhalten sich ebenfalls jederzeit ändern kann. Jede neue Version muss erneut geprüft werden.
- **Keine Garantie:** Die Prüfung wurde nach bestem Wissen und mit der in diesem Bericht beschriebenen Methodik durchgeführt. Eine statische Analyse kann Schadverhalten nicht mit absoluter Sicherheit ausschließen (siehe Abschnitt 8).
- **Keine Haftung:** Die Nutzung der Mod erfolgt auf eigene Verantwortung. Für Schäden, die aus der Nutzung der Mod oder aus dem Vertrauen auf diesen Bericht entstehen, wird keine Haftung übernommen.

11. Kontakt

Fragen, Hinweise oder Korrekturen zu diesem Bericht an Riyu(FPS):

Kanal	Kontakt
Discord	@RiyuFPS
GitHub	https://github.com/ryufps
E-Mail	JulianMink.2007@gmail.com

Anhang A: Prüfsummen (SHA-256)

Website-Dateien und Gradle-Wrapper:

Datei	SHA-256
bauloo-styles-1.6.0-mc1.21.11-auto-update.jar	20684bceb237774ae1bc9b208216b057d3f93fbebdf9b4cfa72cdb5856d6c19f
bauloo-styles-1.6.0-mc1.21.11.jar	3056d0be0cb5e4daa3d3b7830be551036a0e4c3b4976e922fe1b531cafa23b7a
gradle/wrapper/gradle-wrapper.jar	cb0da6751c2b753a16ac168bb354870ebb1e162e9083f116729cec9c781156b8

Release-JARs im Cloud-Repository:

Datei (release/...)	SHA-256
bauloo-styles-no-auto-update.jar	3056d0be0cb5e4daa3d3b7830be551036a0e4c3b4976e922fe1b531cafa23b7a
bauloo-styles.jar	20684bceb237774ae1bc9b208216b057d3f93fbebdf9b4cfa72cdb5856d6c19f
mc/1.21.4/bauloo-styles-no-auto-update.jar	98799a3611816185f989dfeb5f2e856152bd61f128bbdba6ab28295fb945d90f
mc/1.21.4/bauloo-styles.jar	e50cd4aba5771e01aba40794c66e40773c8d4122189dca766411c47350c31426
mc/1.21.5/bauloo-styles-no-auto-update.jar	ca4a51a949659800bb401c04642578c471f552e798f7f7a8c6ad7201f3287a1c
mc/1.21.5/bauloo-styles.jar	352ea6774ff857b8bb8da3a6029ba6da4213a695659406fd349a1bceac1f346a
mc/1.21.6/bauloo-styles-no-auto-update.jar	8ff1ed4cc65a835f91bdb4deadadfbe38596c58b5711fbb71df1fa56783c5ad6
mc/1.21.6/bauloo-styles.jar	25687de1463f1f4c63eee9b24f62d8446177e268325ae9a9870fc5b1d11ea26f
mc/1.21.7/bauloo-styles-no-auto-update.jar	577269b2327aee2830406c5fcc2d825e1f315f2b2a71f46544b06239e726ea05
mc/1.21.7/bauloo-styles.jar	a97b9f4d9cbc302601ed255458220f24c2d4ea4bd1b3ba1dd01027325110c4c9
mc/1.21.8/bauloo-styles-no-auto-update.jar	541a1da7d4f84f4fd4bff388d77a65cc08344d971d0b868f6828d6bc6bd0e08d
mc/1.21.8/bauloo-styles.jar	d41c76e2d371739ddd45204ab8e1d044fe5d4394a2f1e5046fcd4151d5630636
mc/1.21.9/bauloo-styles-no-auto-update.jar	58f5236cd44c2aacec83d43990bfd0450f2822397a0542c48fe1f89132517608
mc/1.21.9/bauloo-styles.jar	1b960c95129934e368a53180c03e29bdb9d9ffb11c30050a66bd9c24953ccd20
mc/1.21.10/bauloo-styles-no-auto-update.jar	46ab9ea644baa1f9176182cfd0d456c2e83cf54d1d2aeada60c3ced7bcc2df65
mc/1.21.10/bauloo-styles.jar	b2b3f3689e7ae54a94349ac222b645a5c03916c8ea119e8cbfb44b86c85b0545
mc/26.1/bauloo-styles-no-auto-update.jar	5e8a8db1fab6a6da6b6c5f88435f312527b6ae98dad304cebb9932572e4bb32c
mc/26.1/bauloo-styles.jar	79c9d928c93e0a02da256a37fe79312565032fc801e4ec8202a71a88722236ba
mc/26.1.1/bauloo-styles-no-auto-update.jar	e3cd2756d9dc0cc5082c9d9cfacc2e9e918b0ac742e5b34f8b708521653ec518
mc/26.1.1/bauloo-styles.jar	177ad54b6940111db0b0b75de644e07d16db9aa99524cb5e9c711302b656cfc5
mc/26.1.2/bauloo-styles-no-auto-update.jar	65dc8b16a342305f2410ae194e0b7bd7d3cf58df21bda743c8904dab966d4c04
mc/26.1.2/bauloo-styles.jar	53c80483bf53dd22b60b1a578ade8baf04a21e78fc5704f3004eb722b5f2c371
mc/26.2/bauloo-styles-no-auto-update.jar	d92cce82f85dfab96925f75541ef1f06c77c4a6c36ba91c60cd2146fca33e206
mc/26.2/bauloo-styles.jar	3aab151031a5f7a58a40ea977063a0f28257b69f5258c8e4b7650b2ad13dd528

Anhang B: Werkzeuge und Nachvollziehbarkeit

Werkzeug	Version / Herkunft
Vineflower (Decompiler)	1.11.1 von Maven Central, SHA-1 77767cdbc76d116c6d5e1565ec387e531085b9f4 (verifiziert)
JDK	Eclipse Adoptium OpenJDK 21.0.12
Gradle / Fabric Loom	9.6.1 / 1.17.16 (aus dem Repository, isolierter Gradle-Cache)
Analyse-Skripte	Eigene Python-Skripte (Konstanten-Pool-Parser, JAR-Vergleich, Pack-Scanner, .pyc-Vergleich), ausgeführt mit python -I; kein untersuchter Code wurde geladen